

TraGraph-GS: Trajectory Graph-based Gaussian Splatting for Arbitrary Large-Scale Scene Rendering

Xiaohan Zhang^{ID}, Sitong Wang^{ID}, Yushen Yan^{ID}, Yi Yang^{ID}, Mingda Xu^{ID}, and Qi Liu^{ID}

Abstract—High-quality novel view synthesis for large-scale scenes presents a challenging dilemma in 3D computer vision. Existing methods typically partition large scenes into multiple regions, reconstruct a 3D representation using Gaussian splatting for each region, and eventually merge them for novel view rendering. They can accurately render specific scenes, yet they do not generalize effectively for two reasons: (1) rigid spatial partition techniques struggle with arbitrary camera trajectories, and (2) the merging of regions results in Gaussian overlap to distort texture details. To address these challenges, we propose TraGraph-GS, leveraging a trajectory graph to enable high-precision rendering for arbitrarily large-scale scenes. We present a spatial partitioning method for large-scale scenes based on graphs, which incorporates a regularization constraint to enhance the rendering of textures and distant objects, as well as a progressive rendering strategy to mitigate artifacts caused by Gaussian overlap. Experimental results across four aerial, four ground, and two cross-view (aerial-to-ground) datasets demonstrate the superior performance and remarkable efficiency of our method. Notably, utilizing only a single 24 GB GPU, our approach achieves average PSNR improvements of 0.36 dB, 1.24 dB, and 0.44 dB on large-scale aerial, ground, and cross-view datasets, respectively, compared to state-of-the-art approaches.

Index Terms—Novel view synthesis, arbitrary large scenes, 3D Gaussian splatting, trajectory graph.

I. INTRODUCTION

NOVEL view synthesis has evolved from geometry-based methods [1] and light field rendering [2] to neural representations. NeRF [3] marked a breakthrough with continuous volumetric functions, inspiring extensions in anti-aliasing [4], unbounded modeling [5], and training acceleration [6]. However, neural field methods remain limited by memory and computational demands. More recently, 3D Gaussian Splatting (3DGS) [7] introduced an explicit representation using Gaussian primitives with efficient rasterization, achieving high visual

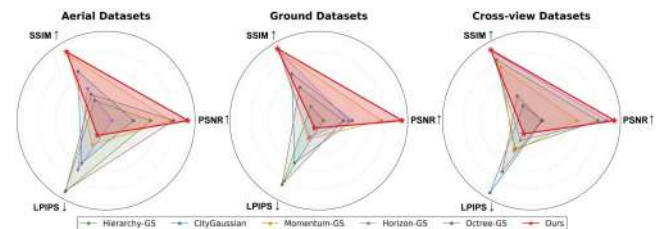


Fig. 1. Quantitative comparison on Aerial, Ground, and Cross-view datasets. Our TraGraph-GS (red stars) outperforms existing methods across all metrics (PSNR $\uparrow$, SSIM $\uparrow$, and LPIPS $\downarrow$), showing balanced performance and seamless adaptability to various camera trajectories.

quality and real-time rendering, with subsequent extensions to diverse scenes [8], [9].

For large-scale scenes, processing all data simultaneously requires enormous GPU memory, making divide-and-conquer approaches particularly promising. These methods [10][11][12][13][14] supported parallel processing of individual chunks followed by consolidation, which are capable of handling scenes spanning several kilometers with thousands of input images with preserved real-time rendering performance. *Large-scale scenes encompass both large-scale aerial and ground scenes. However, achieving high-quality novel view synthesis on both types of datasets simultaneously presents a significant challenge.* Due to the different camera trajectories in aerial and ground datasets, the spatial partitioning approaches of previous large-scale scene rendering methods are not generally applicable to both types. The division of cameras and sparse point clouds produced by Structure-from-Motion (SfM) into multiple regions, known as spatial partitioning, significantly impacts the quality of scene rendering. Aerial datasets are typically captured with uniform, ideal camera paths, whereas ground datasets suffer from non-uniform trajectories due to road constraints. Moreover, aerial datasets focus on distant objects, whereas ground datasets prioritize near-field details.

Consequently, previous spatial partitioning strategies relying on sparse point clouds or camera distribution lack the generalization capability across these diverse dataset types. Specifically, methods like CityGaussian [10], Hierarchy-GS [12], and Momentum-GS [15] typically partition the scene into a uniform $m \times n$ grid based on the spatial distribution of point clouds and assign nearby cameras to each block. As illustrated in Fig. 2(a), for non-uniform camera trajectories, this approach results in an imbalanced distribution of cameras across regions (e.g., $R1$ vs. $R3$) and fails to effectively minimize GPU memory consumption. To address trajectory irregularities,

Received 10 June 2025; revised 12 April 2026; accepted 25 June 2026. Date of publication 2 July 2026; date of current version 4 September 2026. This work was supported in part by the GJYC program of Guangzhou under Grant 2024D01J0081, in part by the ZJ program of Guangdong under Grant 2023QN10X455, and in part by the Fundamental Research Funds for the Central Universities under Grant 2025ZYGXZR053. Recommended for acceptance by V. B. Radhakrishnan. (Xiaohan Zhang and Sitong Wang contributed equally to this work.) (Corresponding author: Qi Liu.)

The authors are with the School of Future Technology, South China University of Technology, Guangzhou 511442, China (e-mail: ftxiaohanzhang@mail.scut.edu.cn; ftkanewang@mail.scut.edu.cn; 202364871411@mail.scut.edu.cn; ftyy@mail.scut.edu.cn; ftxumingda@mail.scut.edu.cn; drliuqi@scut.edu.cn).

This article has supplementary downloadable material available at <https://doi.org/10.1109/TPAMI.2026.3709368>, provided by the authors.

Digital Object Identifier 10.1109/TPAMI.2026.3709368

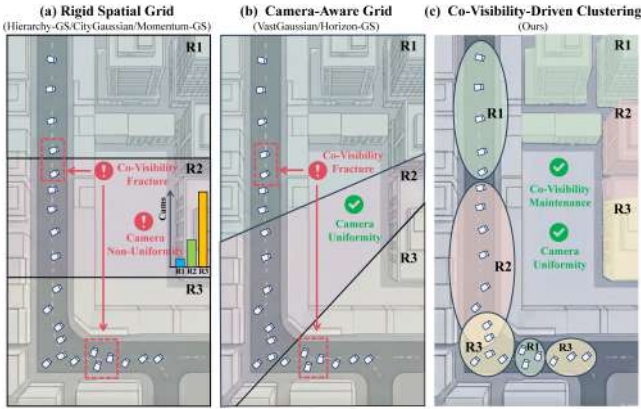


Fig. 2. Comparison of different scene partitioning strategies. The scene is divided into three regions $R1$, $R2$, and $R3$, each assigned a dedicated subset of training cameras for distributed optimization. (a) Rigid Spatial Grid: Employs a fixed grid based on point cloud distribution, leading to significant camera non-uniformity and co-visibility fractures where related cameras are separated by rigid boundaries. (b) Camera-Aware Grid: Achieves camera uniformity by considering camera density, but still suffers from co-visibility fractures due to the inherent limitations of rectangular spatial partitioning. (c) Co-Visibility-Driven Clustering: Groups cameras based on shared visibility rather than pure spatial proximity. This strategy ensures co-visibility maintenance and adaptively clusters cameras according to the actual trajectory topology.

approaches such as VastGaussian [11] and Horizon-GS [16] project camera positions onto the ground plane to divide the space. While this distribution-aware partitioning achieves better camera uniformity (see Fig. 2(b)), two fundamental issues remain unresolved across these grid-based methods: first, cameras sharing significant co-visibility are often isolated into separate blocks—a phenomenon we term “co-visibility fracture”—which degrades novel view rendering performance; second, when local representations are merged into a unified global Gaussian, the resulting Gaussian stacking and overlap induce texture distortion and floating artifacts.

To address these limitations, we propose TraGraph-GS, a trajectory graph-based Gaussian splatting method for high-precision rendering of both aerial and ground scenes. In contrast to previous rigid or camera-aware grids, our co-visibility-driven clustering strategy (Fig. 2(c)) effectively maintains co-visibility while preserving camera uniformity. Fig. 1 illustrates the rendering results on various datasets across different scene types, where our method consistently achieves the state-of-the-art (SOTA) performance. Our approach constructs an image connectivity graph where images serve as vertices and feature matching counts as edge weights. It then applies graph segmentation algorithms to partition cameras and sparse point clouds into regions. This visibility-based partitioning strategy adapts to arbitrary camera trajectories, ensuring consistent reconstruction quality. We leverage the connectivity properties of the graph to apply multi-view constraints to disordered images, enhancing texture detail rendering, and design multi-scale Gaussians to improve distant scene rendering. Ultimately, we employ a progressive rendering approach that prioritizes regions by their contribution to novel view synthesis. In detail, primary regions handle main rendering tasks while auxiliary regions dynamically support areas with insufficient coverage. Our contributions are summarized as follows:

- We propose a trajectory graph-based spatial partitioning method to adaptively divide scenes into various sub-regions, which generalizes well across arbitrarily large-scale scenes with diverse camera trajectories.
- For disordered image collections, our trajectory graph identifies neighboring images for each viewpoint, enabling us to design effective multi-view constraints and multi-scale Gaussian representations to improve both fine texture details and distant scene rendering quality.
- A progressive rendering strategy is introduced to avoid the floating points typically caused by the merging of local Gaussians, resulting in more coherent and visually pleasing renderings.

II. RELATED WORK

A. Novel View Synthesis

Novel view synthesis has evolved from geometry-based methods [1] and light field rendering [2] to neural representations. NeRF [3] marked a breakthrough by representing scenes as continuous volumetric functions, inspiring extensions in anti-aliasing [4], [5], unbounded modeling [17], and training acceleration [6], [18]. Further research explored stratified representations for occlusions [19], large-scale rendering [20], and appearance decoupling [21]. Despite these advances, neural field methods remain computationally demanding for large scenes. More recently, 3D Gaussian Splatting (3DGS) [7] introduced an explicit representation with efficient rasterization, achieving high visual quality and real-time rendering. Subsequent works have further improved 3DGS via compression [22], [23], anti-aliasing [8], and relightable rendering [24].

B. Large-Scale Ground Scene Rendering

For large-scale ground scene rendering, divide-and-conquer approaches that partition scenes into chunks have shown significant promise. As these methods enable parallel processing of individual chunks followed by consolidation, they show excellent performance on handling large-scale scenes across kilometers with thousands of input images. For example, Block-NeRF [25] partitioned the scene into parts with 50% overlap and computed a representation for each part. Progressive propagation techniques have proven effective for improving geometric accuracy in ground scene rendering. These approaches leveraged patch matching and multi-view consistency checks to propagate accurate geometric information from well-modeled regions to under-modeled regions [26]. By incorporating planar constraints and geometric filtering, they produced more compact and accurate scene representations, particularly beneficial for the texture-less regions commonly found in ground scenes. Level-of-detail (LOD) management techniques are essential for efficiently rendering massive ground scenes. Hierarchical approaches construct tree-based structures with interior and leaf nodes, defining efficient methods to merge primitives and determine appropriate cuts through the hierarchy based on projected screen size [27]. LOD variants were proposed for point-based

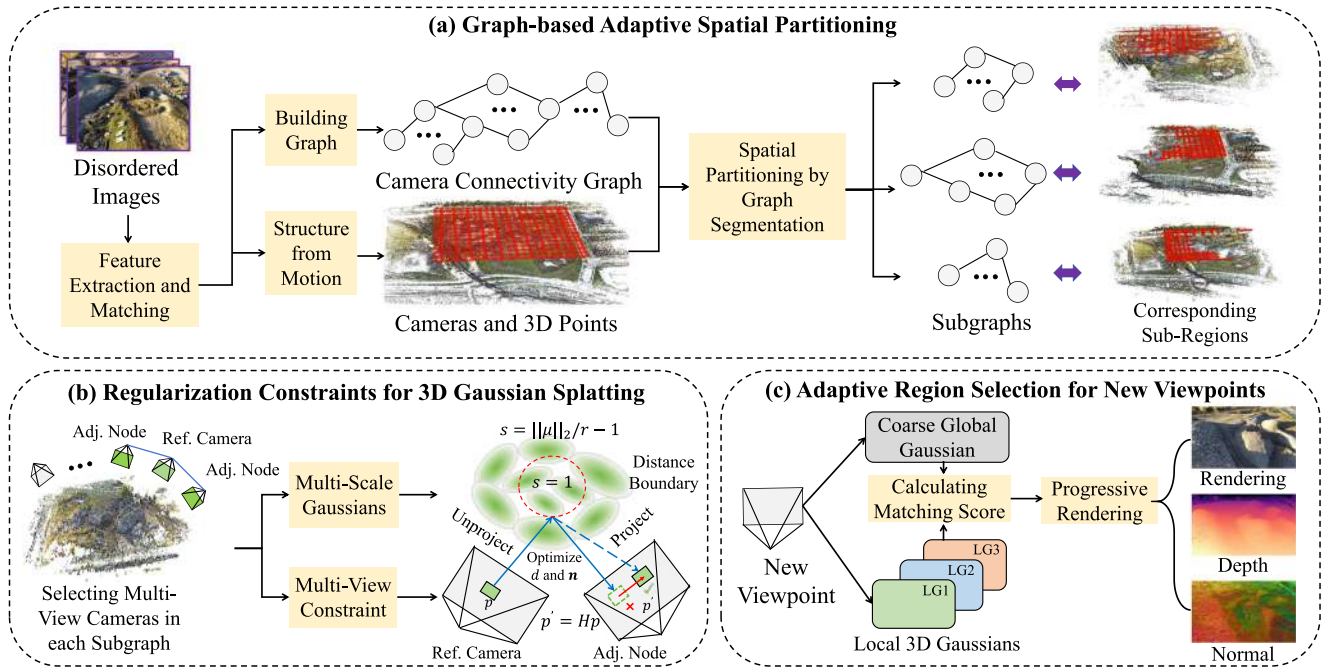


Fig. 3. Pipeline of our TraGraph-GS: a) Scene Partitioning: From unordered images, we first extract features and construct a camera connectivity graph (images as vertices, feature matching counts as edges' weights). Simultaneously, Structure-from-Motion (SfM) provides camera poses and sparse 3D points. Based on this spatial structure, both cameras and points are then partitioned into multiple regions using graph methods. b) Local Training: Building on this partitioning, each region trains local Gaussians independently. During training, neighborhood cameras are identified through subgraphs, where multi-view constraints enhance texture details and multi-scale Gaussians improve distant scene rendering quality. c) Adaptive Rendering: For new viewpoint synthesis, we compute matching scores between global and local Gaussians, thereby ranking them by contribution. Accordingly, high-contribution local Gaussians render primary areas, while low-contribution ones supplement insufficient regions.

representations [28] [29], but these methods cannot be rasterized. Kerbl et al. [12] introduced a hierarchical 3D Gaussian representation that preserved visual quality for large scenes while providing an efficient LOD solution for rendering distant content. Similarly, Octree-GS [30] integrated an explicit octree structure to manage multi-scale Gaussian primitives, enabling dynamic Level-of-Detail (LOD) selection and consistent real-time rendering by culling redundant computations in zoomed-out views. Beyond structural optimization, On-the-Fly [31] addresses the efficiency bottleneck of long-duration captures by interleaving SLAM-based tracking with incremental mapping, eliminating the need for offline Structure-from-Motion (SfM). Techniques focused on anti-aliasing at different scales [32] further enhanced ground scene rendering quality across varying viewing distances.

C. Large-Scale Aerial Scene Rendering

Large-scale aerial scene reconstruction has evolved from early SfM systems [33], [34] to efficient neural rendering. For large-scale reconstruction, divide-and-conquer strategies [35], [36] partition scenes into spatially adjacent blocks to enable distributed computing and overcome memory limitations. Typically, each block is trained independently before consolidation. To handle lighting and exposure variations across captures, several methods incorporate appearance encoding [37], effectively reducing reconstruction artifacts. For city-scale aerial rendering, methods like CityGaussian [10] achieved impressive visual

results through its LoD-based rendering. Subsequent work, CityGaussianV2 [38], further optimized the geometric fidelity and system throughput for larger-scale data, although at the cost of some fine-grained visual details that were more prominently captured in the initial version CityGaussian [10]. VastGaussian [11] introduced progressive partitioning for large scenes and decoupled appearance modeling to handle lighting variations in aerial data. To enhance training stability, Momentum-GS [15] introduces a momentum-based self-distillation framework that ensures spatial consistency and eliminates lighting discrepancies across independently trained blocks. Furthermore, to bridge the gap between different perspectives, Horizon-GS [16] proposed a unified framework for large-scale aerial-to-ground reconstruction, employing a coarse-to-fine training strategy to resolve viewpoint discrepancies and gradient conflicts between aerial and street-level imagery.

III. METHOD

In this paper, we propose TraGraph-GS, a high-quality rendering method for arbitrary scenes. The pipeline of our method is shown in Fig. 3. Given disordered scene photographs as input, we first design a graph-based adaptive spatial partitioning approach (Section III-A) which constructs an image connectivity graph through feature matching and utilizes graph segmentation for adaptive spatial partitioning. Another benefit of using a graph structure is establishing connection relationships between disordered images. We then employ Regularization Constraints for

Algorithm 1: BFS-Based Graph Segmentation.

Input: Camera connectivity graph $G(V, E, w)$, number of segmented regions k .
Definition: Nodes in each region $\{P_i\}_{i=1}^k$, unassigned nodes U .
initialize $U = V$
for $i = 1, \dots, k$ **do**
 $P_i = \emptyset$
 seed node $n = \operatorname{argmax}_{u \in U} (\sum_{(u,v) \in E} w(u, v))$
 add n **to** P_i
 while $|P_i| < \frac{|V|}{k}$:
 adjacent nodes
 $N = \{v \in V \setminus P_i \mid (u, v) \in E, u \in P_i\}$
 update $P_i = P_i \cup N, U = U - N$
 for u **in** U **do**
 score $s_i(u) = \sum_{v \in P_i} w(u, v)$
 score median $m = \operatorname{median}(\{s_i(u)_{i=1}^k\})$
 high-weight regions $H = \{i \mid s_i(u) > m\}$
 assign nodes $u \in P_i, i = \operatorname{argmin}_{i \in H} |P_i|$

3D Gaussian Splatting (Section III-B) to enhance texture details and improve the rendering of distant scenes. Finally, we apply the adaptive region selection for new viewpoints (Section III-C) to avoid floating artifacts caused by Gaussian overlap in previous methods.

A. Graph-Based Adaptive Spatial Partitioning

Building Camera Connectivity Graph: Previous methods [10] [11] [39] merely used point clouds and camera poses obtained through feature extraction and matching [40], and SfM [41] to divide scenes into multiple regions, while neglecting the structural information of the feature correspondences between images. We fully leverage feature matching information by constructing a camera connectivity graph where images serve as vertices and feature matching counts determine edge weights. The camera connectivity graph is defined as $G = (V, E, w)$, where $V = \{v_1, v_2, \dots, v_n\}$ is the set of vertices (or nodes), $E = \{(v_i, v_j) \mid v_i, v_j \in V\}$ is the set of edges connecting pairs of vertices, and $w : E \rightarrow \mathbb{R}$ is a weight function that assigns a real number weight to each edge. Higher edge weights indicate more extensive co-visibility regions between image pairs. To filter out erroneous matches and improve computational efficiency, we retain only those edges whose weights exceed the median of the edge weight distribution. This graph simplification has a negligible impact on rendering quality while preserving graph connectivity, as verified in the supplementary material.

Graph Segmentation based on BFS: Various methods can decompose a graph into multiple subgraphs, such as spectral decomposition-based [42] and Kernighan-Lin-based [43] graph segmentation approaches. However, Spectral decomposition exhibits inherent limitations in achieving balanced node distribution, which can lead to potential memory overflow issues. Conversely, Kernighan-Lin algorithms fail to preserve the connectivity of nodes in regions, thereby compromising rendering fidelity. We design a BFS-based graph segmentation method that

simultaneously ensures connectivity among nodes within each subgraph while maintaining balanced node distribution across regions. First, we identify the node n with the highest degree among all unassigned nodes and designate it as the seed node. Employ the breadth-first search (BFS) [44] approach to add nodes until the region contains $|V|/k$ nodes, thus establishing these nodes with edges as a region. Dividing the graph into k regions in this way, subsequently, for each remaining unassigned node, calculate its score s_i relative to each existing region. To maintain balance in the node distribution across regions, assign each unassigned node to the region with the fewest nodes among those regions whose scores exceed the median score for that node. The detailed algorithm is outlined in Algorithm 1.

Computational Overhead Analysis: Previous large-scale rendering methods typically employ spatial partitioning based on either the distribution of the sparse point cloud ($O(N_p)$ complexity, where N_p is the total number of sparse 3D points) or the geometric positions of cameras ($k \cdot O(N_{cams})$ complexity, where N_{cams} is the total number of input cameras and k denotes the number of regions). While these methods are computationally efficient, they often fail to account for the underlying scene topology, leading to imbalanced GPU loads or co-visibility fractures. In contrast, TraGraph-GS constructs the camera connectivity graph by directly reusing the feature-matching results that are already produced during the SfM stage—a mandatory prerequisite shared by all 3DGS-based methods—thereby introducing virtually no additional computational cost for graph construction. Building upon this graph, our BFS-based segmentation operates with a complexity of $O(k \cdot (|N_{cams}| + |E|))$, where $|E|$ represents the validated feature-matching edges. Although our approach explicitly considers the connectivity ($|E|$), the overhead remains negligible. The wall-clock time of BFS-based segmentation for each dataset is reported in the supplementary material.

B. Regularization Constraints for 3DGS

3D Gaussian Splatting: We implement parallel training of local Gaussians for each region to optimize GPU memory utilization. 3DGS [7] represents scenes using anisotropic 3D Gaussians with learnable attributes. Each Gaussian $G(\mathbf{x})$ is defined as:

$$G(\mathbf{x}) = e^{-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu})} \quad (1)$$

where $\boldsymbol{\mu} \in \mathbb{R}^{3 \times 1}$ represents the mean vector (position), and $\boldsymbol{\Sigma} \in \mathbb{R}^{3 \times 3}$ represents the covariance matrix that defines the shape and orientation of the Gaussian. To ensure the positive semi-definite property of the covariance matrix during optimization, it's typically expressed as $\boldsymbol{\Sigma} = \mathbf{R} \mathbf{S} \mathbf{S}^T \mathbf{R}^T$, where the rotation matrix $\mathbf{R} \in \mathbb{R}^{3 \times 3}$ is orthogonal, and the scale matrix $\mathbf{S} \in \mathbb{R}^{3 \times 3}$ is diagonal. For rendering, the color of each pixel $\mathbf{p}$ is calculated by blending N ordered Gaussians $\{G_i \mid i = 1, \dots, N\}$ that overlap $\mathbf{p}$:

$$\mathbf{c}(\mathbf{p}) = \sum_{i=1}^N \mathbf{c}_i \alpha_i \prod_{j=1}^{i-1} (1 - \alpha_j) \quad (2)$$

where α_i is a learned opacity value, and $\mathbf{c}_i$ is the learned color of Gaussian G_i .

Multi-View Constraint: While 3DGS achieves impressive rendering quality, it often struggles with accurate geometry reconstruction, particularly in textureless regions. To address this issue, a multi-view constraint can be introduced based on patch matching techniques. First, for disordered images, we can identify the neighborhood frames of each image from the subgraphs. Second, depth and normal maps are rendered for each image by overlapping the positions and orientations of 3D Gaussians using alpha blending. Then, for each pixel p in a reference view, a spatial plane $(d, \mathbf{n})$ is established where $\mathbf{n} \in \mathbb{R}^{1 \times 3}$ represents the pixel's rendered normal and d indicates the distance from the camera coordinate origin to the spatial plane. For a pixel $\mathbf{p}$ in the reference view, we can use homography warping to project it to its corresponding pixel $\mathbf{p}'$ in a neighborhood view:

$$\mathbf{p}' = \mathbf{H}\mathbf{p} \quad (3)$$

in which $\mathbf{H}$ is defined as:

$$\mathbf{H} = \mathbf{K} \left(\mathbf{R} - \frac{\mathbf{t}\mathbf{n}^T}{d} \right) \mathbf{K}^{-1} \quad (4)$$

$\mathbf{R} \in \mathbb{R}^{3 \times 3}$ and $\mathbf{t} \in \mathbb{R}^{1 \times 3}$ represent the relative rotation and translation between the reference and source views, and $\mathbf{K} \in \mathbb{R}^{3 \times 3}$ is the camera's intrinsic matrix. To mitigate the susceptibility of single-pixel-based scene geometry optimization to influences such as lighting variations, we construct a patch $\mathbf{P}_i$ by selecting several neighboring pixels around each considered pixel [45]. We evaluate the color consistency between corresponding patches using Normalized Cross Correlation (NCC) [46] and incorporate this measure into our optimization objective:

$$L_{mv} = \sum_{\mathbf{p} \in \mathbf{P}_i} (1 - \text{NCC}(\mathbf{p}, \mathbf{p}')) \quad (5)$$

Our final training loss L is defined as:

$$L = (1 - \lambda)L_1 + \lambda L_{SSIM} + L_{mv} \quad (6)$$

where image reconstruction loss functions L_1 and L_{SSIM} are illustrated in 3DGS [7].

Multi-Scale Constraint: For large-scale scenes with varying distances, standard Gaussian representation often struggles with distant objects. To address this, a position-aware point adaptive control [47] can be employed, applying different scales to Gaussians based on their distance from the viewer:

$$\gamma(\boldsymbol{\mu}) = \begin{cases} 1 & \text{if } \|\boldsymbol{\mu} - \mathbf{c}\|_2 < 2r \\ \|\boldsymbol{\mu} - \mathbf{c}\|_2 / r - 1 & \text{if } \|\boldsymbol{\mu} - \mathbf{c}\|_2 \geq 2r \end{cases} \quad (7)$$

where r denotes the scene radius and $\mathbf{c} \in \mathbb{R}^{3 \times 1}$ denotes the center of cameras. When the directional gradient of a Gaussian ellipsoid exceeds the predetermined threshold, we implement a specific protocol. For cases where the maximum scale of Gaussian ellipsoid $\max(s) < g \cdot \gamma(\boldsymbol{\mu})$ is satisfied, we perform a cloning operation on the Gaussian ellipsoid. Conversely, when this criterion is not met, we execute a splitting procedure instead. Additionally, we eliminate Gaussian ellipsoids where $\max(s) > b \cdot \gamma(\boldsymbol{\mu})$, with g and b serving as scale threshold parameters that

govern these operations. This approach uses larger Gaussians for distant objects and smaller ones for nearby regions, improving the rendering quality across different scales.

To further enhance the reconstruction of distant views and regions with sparse camera trajectories, we adopt the max-densification strategy proposed in Hierarchy-GS [12]. Unlike traditional methods that rely on average gradients which can be overly diluted in under-observed areas, this strategy captures fine-grained geometric signals by utilizing a densification metric ∇_{max} . This metric weights the view-space position gradient $\nabla_{\mu_{2D}}$ —which represents the sensitivity of the image-space loss with respect to the 2D projected center of each Gaussian primitive—by the Gaussian's maximum screen-space radius r_{max} and its opacity α :

$$\nabla_{max} = |\nabla_{\mu_{2D}}| \cdot 2 \cdot r_{max} \cdot \alpha^{1/5} \quad (8)$$

The rationale for adopting this formulation is that in far-field or sparsely sampled regions, standard gradient-based densification often fails because the small visual footprint of distant Gaussians leads to vanishingly small average gradients due to perspective attenuation. By incorporating r_{max} , the strategy compensates for this attenuation, effectively amplifying the densification signal for distant primitives that would otherwise be ignored. Furthermore, the inclusion of $\alpha^{1/5}$ ensures that semi-transparent candidates with high potential for geometric refinement are prioritized. By comparing ∇_{max} against a threshold, we force the generation of more primitives in texture-less or sparsely sampled far-field regions.

However, the max-densification strategy may lead to an excessive number of Gaussian primitives. To maintain a manageable GPU memory footprint, we incorporate a split generation constraint. Each Gaussian primitive tracks its evolution through a split counter N_{gen} . A splitting procedure is performed only if N_{gen} satisfies:

$$N_{gen} < \max \left(1, \frac{N_{max}}{\gamma(\boldsymbol{\mu})} \right) \quad (9)$$

Where N_{max} is a hyperparameter that specifies the maximum number of splits permitted for each Gaussian primitive, serving as a memory budget control to curb excessive growth in the total number of primitives, and $\gamma(\boldsymbol{\mu})$ is the distance-dependent scaling factor calculated by (7). This mechanism explicitly reduces the allowable split counts as the distance from the center increases. The rationale behind this design is two-fold: (1) **Near-field Gaussians are small and densely packed**, capturing fine-grained texture details. The relaxed split constraint in close-range regions allows sufficient primitive generation to faithfully reconstruct surface appearance. (2) **Far-field Gaussians are large and sparsely distributed**, providing stable and artifact-free coverage of distant regions. Limiting allowable splits for far-field primitives prevents redundant tiny Gaussian accumulation, suppressing floating artifacts. Consequently, this strategy preserves high-fidelity rendering for both aerial and ground scenes.

Furthermore, the multi-scale Gaussian representation is particularly vital for ground-level datasets, which typically encompass extensive depth ranges. First, as $\gamma(\boldsymbol{\mu})$ is defined as a

continuous function of distance, it guarantees the continuity of Gaussian attribute transitions across space, facilitating smooth visual blending from near-field details to far-field structures. Second, governed by the split generation constraint in (9), the allowable number of splitting operations decreases as the distance from the scene center increases. This mechanism prevents the accumulation of redundant tiny primitives in distant areas, thereby suppressing floating artifacts and ensuring robust rendering fidelity for large-scale ground trajectories.

C. Adaptive Region Selection for New Viewpoints

Previous methods [10] [11] [12] for large-scale scene rendering combined all local Gaussians into a global Gaussian to render novel viewpoints. However, these approaches often produced floating artifacts at region boundaries due to Gaussian overlap or created holes from improper pruning of local Gaussians. To address these issues, we propose a progressive rendering strategy that prioritizes local Gaussians with higher contributions to the novel viewpoint, followed by supplementary rendering with lower-contribution Gaussians to refine problematic regions. First, we train a coarse global Gaussian representation to capture the overall scene structure. During this global Gaussian training, we downsample both point clouds and images while avoiding the Gaussian ellipsoid densification process, which prevents excessive memory consumption. Next, we render the novel viewpoint using both local and global Gaussians separately, calculating a matching score S for each local-global Gaussian pair:

$$S = (1 - \lambda)L_1 + \lambda L_{SSIM} \quad (10)$$

L_1 and L_{SSIM} are the same as (6). Higher matching scores indicate that the corresponding local Gaussian is more likely to be visible from the novel viewpoint. Then we sort the local Gaussians in descending order $G_1, G_2, \dots, G_k$ based on their matching scores and calculate the corresponding projection opacity on a pixel $o_1(\mathbf{p}), o_2(\mathbf{p}), \dots, o_k(\mathbf{p})$:

$$o_i(\mathbf{p}) = \sum_{j=1}^N \alpha_i \prod_{j=1}^{i-1} (1 - \alpha_j) \quad (11)$$

where α_i is the opacity of the Gaussian ellipsoid that contributes to the pixel $\mathbf{p}$. Finally, we employ a progressive rendering strategy to compute the rendering of pixel $\mathbf{p}$:

$$\mathbf{c}(\mathbf{p}) = \sum_{i=1}^k w_i \mathbf{c}_i(\mathbf{p}) \quad (12)$$

in which $\mathbf{c}_i(\mathbf{p})$ is the color of pixel $\mathbf{p}$ rendered by the local Gaussian G_i and the weight of each local Gaussian w_i is defined as:

$$w_i = \begin{cases} o_i(\mathbf{p}) & \text{if } \sum_{j=1}^{i-1} o_j(\mathbf{p}) < \beta \\ 0 & \text{if } \sum_{j=1}^{i-1} o_j(\mathbf{p}) \geq \beta \end{cases} \quad (13)$$

where β is a hyperparameter, this formula indicates that when the overlapped opacity of local Gaussians with significant contributions at pixel $\mathbf{p}$ is high, it signifies sufficient rendering for

this pixel. Conversely, when the overlapped opacity is low, it suggests insufficient rendering of the pixel, necessitating continued rendering with newly obtained local Gaussians.

IV. EXPERIMENTS

A. Datasets

Aerial Datasets: We evaluate our method on four large-scale publicly available aerial imagery datasets: *Rubble* and *Building* from the Mill-19 dataset [35], and *Residence* and *Sci-Art* from the UrbanScene3D dataset [48]. These four datasets comprise 1,678, 1,941, 2,582, and 3,019 images, respectively.

Ground Datasets: We evaluate our method on four ground-level datasets: *SmallCity*, *Campus*, *Road*, and *General*. *SmallCity* and *Campus* were introduced by [12] and contain 1,500 and 1,190 images, respectively. *Road* and *General* are self-built datasets. *Road* consists of numerous distant objects and complex street scenes captured with a smartphone along an extended and irregular trajectory. This dataset is designed to validate the robustness of algorithms for distant-view reconstruction and the preservation of textural detail. *General* comprises smartphone-captured images of double-layer buildings, which aim at validating the algorithms' robustness for spatial partitioning. *Road* and *General* contain 647 and 619 images, respectively, with a resolution of 1920×1080 .

Horizon-GS Datasets: To further validate the effectiveness of TraGraph-GS in unified aerial-to-ground reconstruction, we incorporate the recently proposed Horizon-GS datasets [16]. This dataset fills a significant gap in the field by providing a rare collection of images where aerial and street-level perspectives are precisely aligned in a unified coordinate system. It encompasses five synthetic and two real-world scenes. In our experiments, we select the scenes with the largest number of images from both the real-world and synthetic datasets to ensure a rigorous evaluation: the real-world scene *Park*, comprising 975 aerial and 5,419 street-view images, and the synthetic scene *Citysample*, which consists of 385 aerial and 1,404 street-view images. These scenes feature diverse environmental conditions and complex camera trajectories. The real-world data was acquired using a DJI M350RTK drone for aerial mapping and a custom-designed camera helmet for street-level traversal.

B. Implementation Details

For four public aerial imagery datasets, our test set selection follows previous methods such as Momentum-GS [15]. Specifically, the *Rubble*, *Building*, *Residence*, and *Sci-Art* scenes contain 21, 20, 21, and 21 test images, respectively. For the ground-level datasets *SmallCity* and *Campus*, we employ a single chunk for both training and test processes, which are consistent with the Hierarchy-GS [12]. For our self-captured datasets, we divide the whole scene into 3 regions and similarly select every 8th image for the test set. The *Road* and *General* test sets comprise 81 and 78 test images, respectively. Regarding the Horizon-GS datasets, we adopt the same testing split as Horizon-GS [16], selecting one image every 32 frames for

TABLE I

COMPARATIVE ANALYSIS OF NEURAL RENDERING AND GAUSSIAN SPLATTING METHODS ACROSS FOUR BENCHMARK DATASETS. THE TABLE PRESENTS QUANTITATIVE RESULTS FOR PSNR, SSIM, AND LPIPS. THE BEST AND SECOND BEST RESULTS ARE HIGHLIGHTED IN **BOLD** AND UNDERLINE, RESPECTIVELY. THE †SYMBOL REPRESENTS THE CASE WHERE APPEARANCE ENCODING IS NOT EMPLOYED.

Metrics	Building			Rubble			Residence			Sci-Art		
	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓	PSNR↑	SSIM↑	LPIPS↓
3DGS [7]	20.46	0.720	0.305	25.47	0.777	0.277	21.44	0.791	0.236	21.05	0.830	0.242
VastGaussian † [11]	21.80	0.728	0.225	25.20	0.742	0.264	21.01	0.699	0.261	22.64	0.761	0.261
Hierarchy-GS [12]	21.52	0.723	0.297	24.64	0.755	0.284	19.97	0.705	0.297	18.28	0.590	0.316
CityGaussian [10]	21.55	0.778	0.246	25.77	0.813	0.228	22.00	0.813	0.211	21.39	0.837	0.230
On-the-Fly [31]	15.12	0.339	0.652	17.27	0.400	0.646	17.03	0.506	0.493	14.40	0.463	0.572
Momentum-GS [15]	23.23	0.815	0.194	25.93	0.827	0.201	22.21	0.818	0.197	23.02	0.856	0.205
Octree-GS [30]	23.66	0.776	0.267	25.34	0.763	0.299	<u>22.29</u>	0.762	0.288	23.38	0.828	0.240
Horizon-GS [16]	22.02	0.773	0.235	24.73	0.823	0.239	20.73	0.753	0.260	21.12	0.807	0.225
Ours	<u>23.30</u>	0.817	0.182	27.35	0.836	0.181	22.82	0.827	0.190	22.63	0.857	0.184

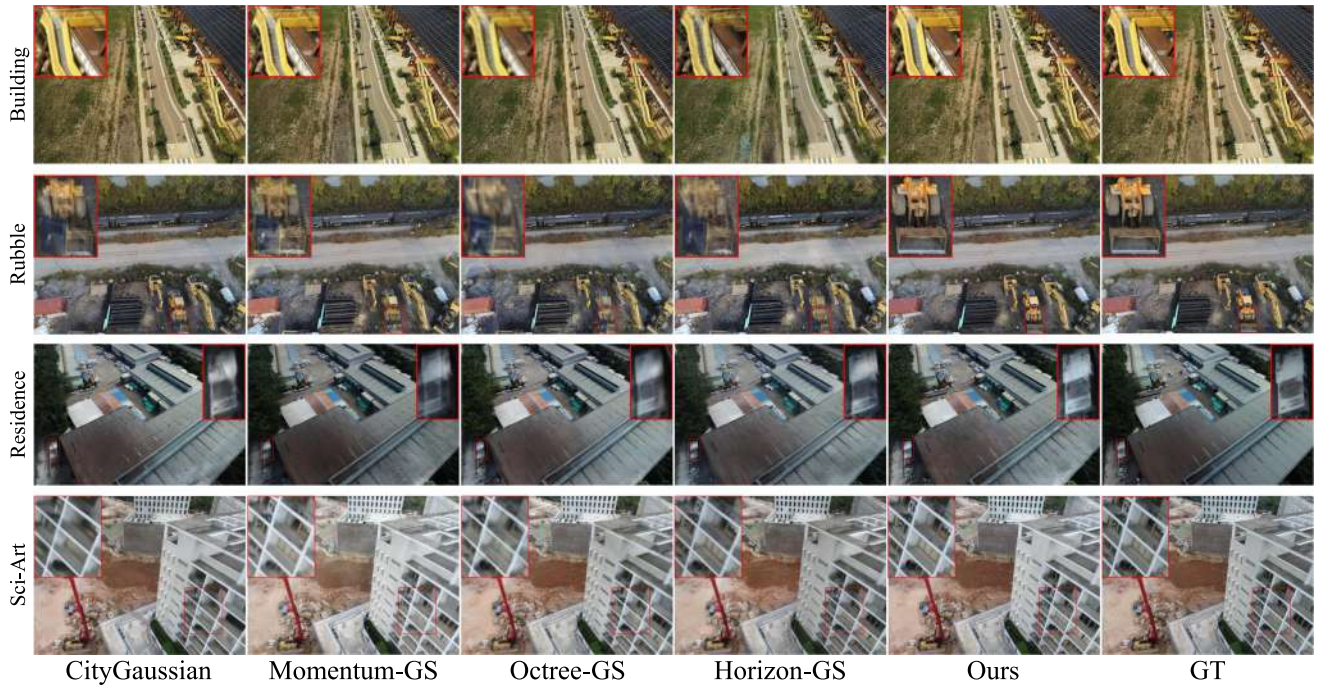


Fig. 4. Qualitative comparison across four aerial datasets highlights the significant challenges of high-fidelity reconstruction in large-scale scenes. Compared to existing methods, our approach demonstrates superior performance in recovering fine-grained textures and maintaining sharp structural clarity for both ground-level objects and distant views.

testing; this results in 200 and 81 test images for the Park and Citysample scenes, respectively. In the multi-view constraint implementation, for each image, we select 4 neighboring cameras with the highest edge weights from the camera connectivity graph. For the multi-scale constraint, we set the scale threshold parameters g and b to 0.01 and 0.1, respectively. Our training consists of 30,000 iterations, with Gaussian pruning applied between iterations 500 and 15,000. The regularization parameter λ in the training loss was set to 0.2. In the progressive rendering process, we use a value of 0.5 for parameter β . All training and test experiments are conducted on a single NVIDIA 4090 GPU with 24 GB memory.

C. Results on Aerial Scenes

Table I presents quantitative results on aerial datasets. Our method achieves the best performance with 24.03 dB average

PSNR, 0.834 SSIM, and 0.184 LPIPS, outperforming Octree-GS by +0.36 dB, Momentum-GS by +0.43 dB, CityGaussian by +1.35 dB, Horizon-GS by +1.88 dB, and Hierarchy-GS by +2.93 dB. Grid-based methods (CityGaussian, Horizon-GS) suffer from co-visibility fractures; Momentum-GS inherits this limitation despite self-distillation; Octree-GS and Hierarchy-GS lack trajectory-aware partitioning; and On-the-Fly struggles with unordered collections due to its reliance on sequential inputs. As shown in Fig. 4, our method faithfully renders distant ground-level textures and far-field structures.

D. Results on Ground Scenes

Table II presents results on ground-level datasets, which pose challenges due to non-uniform trajectories and extensive depth ranges. Existing methods exhibit inconsistent performance: Hierarchy-GS excels on multi-trajectory scenes but fails

TABLE II
COMPARATIVE ANALYSIS OF GAUSSIAN SPLATTING METHODS ACROSS FOUR GROUND DATASETS. THE TABLE PRESENTS QUANTITATIVE RESULTS FOR PSNR, SSIM, AND LPIPS. THE BEST AND SECOND BEST RESULTS ARE HIGHLIGHTED IN **BOLD** AND UNDERLINE, RESPECTIVELY.

Metrics	Small-City			Campus			Road			General		
	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$
3DGS [7]	25.34	0.776	0.337	23.87	0.785	0.378	20.18	0.679	0.397	19.53	0.641	0.411
Hierarchy-GS [12]	26.62	0.820	0.259	24.61	0.807	0.331	14.67	0.405	0.538	20.03	0.655	0.294
CityGaussian [10]	23.68	0.810	0.259	22.82	0.788	0.322	23.88	0.749	0.288	22.61	0.732	0.304
On-the-Fly [31]	18.34	0.596	0.512	14.89	0.580	0.549	18.20	0.529	0.508	16.29	0.535	0.563
Momentum-GS [15]	24.70	0.839	0.229	24.09	0.805	<u>0.309</u>	<u>26.39</u>	0.849	0.174	<u>25.08</u>	0.848	<u>0.134</u>
Octree-GS [30]	22.40	0.758	0.363	22.39	0.737	0.444	22.71	0.677	0.366	23.31	0.743	0.294
Horizon-GS [16]	26.65	0.877	0.176	24.35	0.809	0.310	21.01	0.848	0.165	20.03	0.813	0.169
Ours	27.70	<u>0.873</u>	0.170	24.96	0.820	0.271	26.96	0.849	0.130	25.61	<u>0.839</u>	0.129

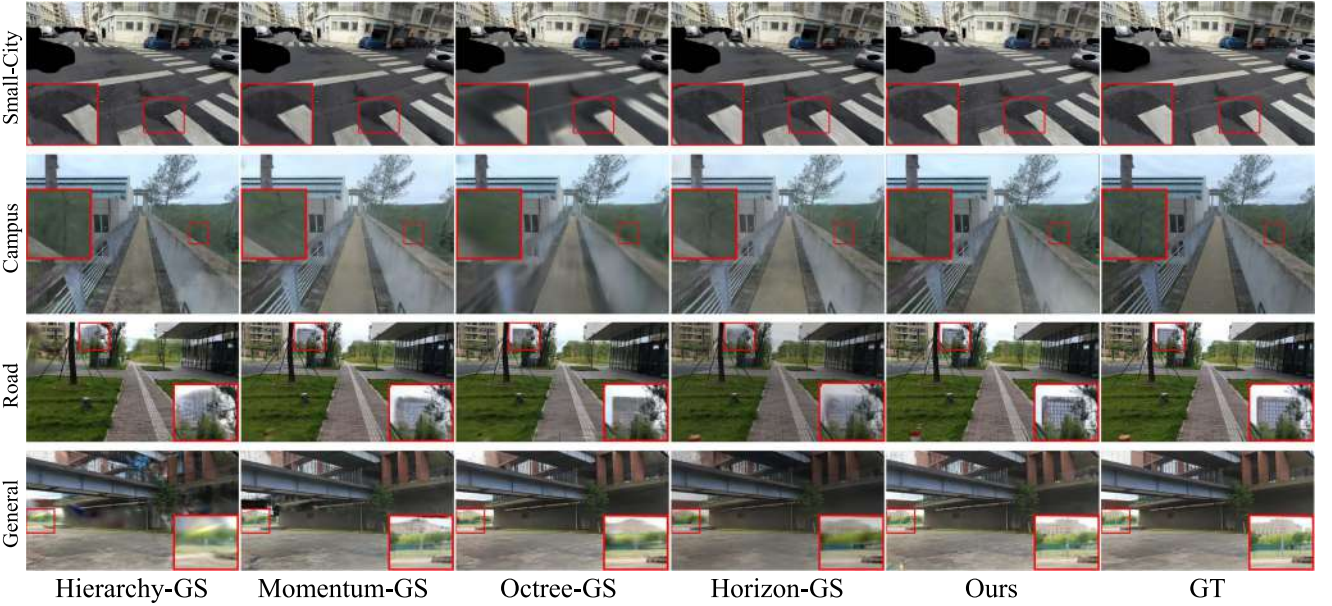


Fig. 5. Qualitative comparison across four ground-level datasets highlights the challenges of high-fidelity rendering in large-scale unbounded scenes. Compared to existing methods, our approach demonstrates superior performance in recovering fine-grained textures and maintaining sharp visual clarity for complex far-field elements.

TABLE III
QUANTITATIVE COMPARISON ON TWO HORIZON DATASETS. WE REPORT PSNR $\uparrow$, SSIM $\uparrow$, AND LPIPS $\downarrow$ FOR BOTH STREET AND AERIAL VIEWS ON EACH SUBSET, AS WELL AS THE CROSS-VIEW AVERAGE. **BOLD** AND UNDERLINE DENOTE THE BEST AND SECOND-BEST RESULTS, RESPECTIVELY.

Method	CitySample						Park						Cross-View Avg.		
	Street			Aerial			Street			Aerial			PSNR	SSIM	LPIPS
	PSNR	SSIM	LPIPS	PSNR	SSIM	LPIPS	PSNR	SSIM	LPIPS	PSNR	SSIM	LPIPS			
CityGaussian [10]	15.44	0.496	0.885	34.02	<u>0.951</u>	<u>0.087</u>	21.98	0.659	0.441	18.91	0.521	0.688	21.62	0.640	0.552
Hierarchy-GS [12]	24.51	0.810	0.298	29.15	0.871	0.206	<u>23.75</u>	<u>0.720</u>	0.294	21.63	0.657	0.427	24.76	0.765	0.306
Momentum-GS [15]	24.53	0.834	<u>0.258</u>	32.20	0.940	0.104	20.37	0.634	0.414	19.87	0.558	0.513	23.60	0.745	0.320
Octree-GS [30]	20.03	0.702	0.390	31.57	0.927	0.122	19.94	0.579	0.540	17.37	0.476	0.678	21.57	0.667	0.434
Horizon-GS [16]	24.28	<u>0.827</u>	0.264	33.95	0.946	0.092	23.14	0.701	0.308	23.85	0.776	0.287	25.26	0.789	0.258
Ours	24.24	0.803	0.230	<u>33.96</u>	0.953	0.081	24.66	0.749	0.244	21.36	<u>0.661</u>	<u>0.356</u>	25.70	0.792	0.223

on single-trajectory ones, while Horizon-GS and CityGaussian struggle with distant views or irregular paths. Our graph-based partitioning adapts naturally to arbitrary trajectories, achieving 26.31 dB average PSNR, 0.845 SSIM, and 0.175 LPIPS, outperforming Momentum-GS by +1.24 dB, CityGaussian by +3.06 dB, Horizon-GS by +3.30 dB, Octree-GS by +3.61 dB, and Hierarchy-GS by +4.83 dB. Fig. 5 confirms high-fidelity rendering of both near-field textures and distant structures.

E. Results on Cross-View Scenes

We evaluate on two Horizon-GS cross-view datasets [16]: real-world Park and synthetic Citysample (Table III). The drastic viewpoint disparity makes this setting particularly challenging. Unlike Horizon-GS, which requires manual aerial-ground separation and two-stage training, our method automatically segregates view types via co-visibility-driven partitioning (Fig. 2(c)).

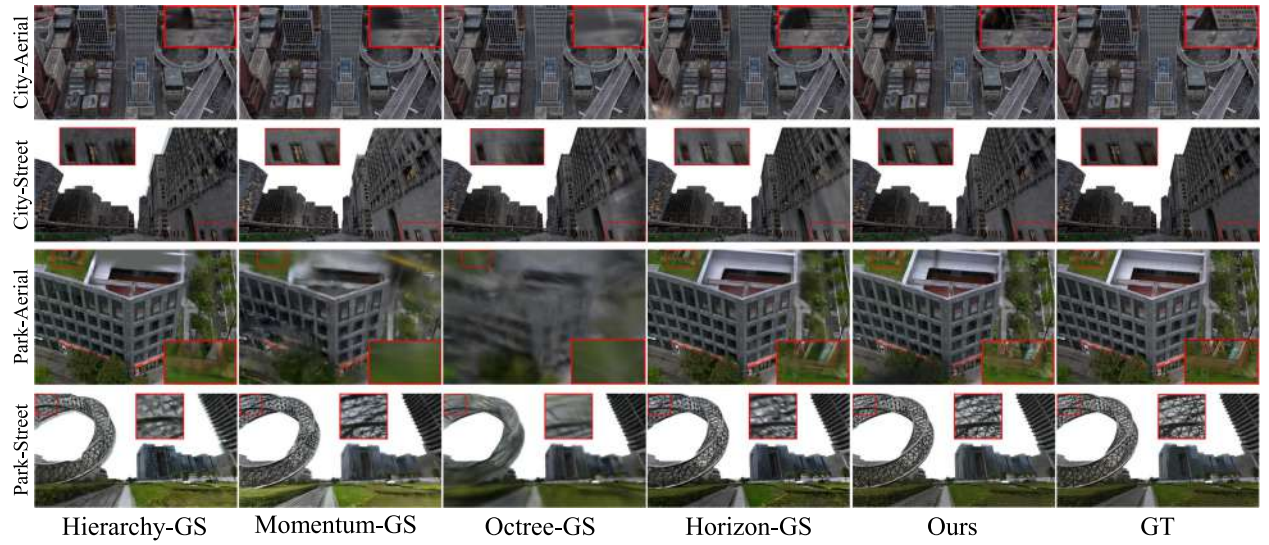


Fig. 6. Qualitative comparison on two Horizon datasets highlights the significant challenges of integrated rendering across aerial and ground-level perspectives. Compared to existing methods, our approach demonstrates superior robustness in fusing cross-view data, achieving enhanced structural integrity and sharper textural details across diverse urban and natural scenes.

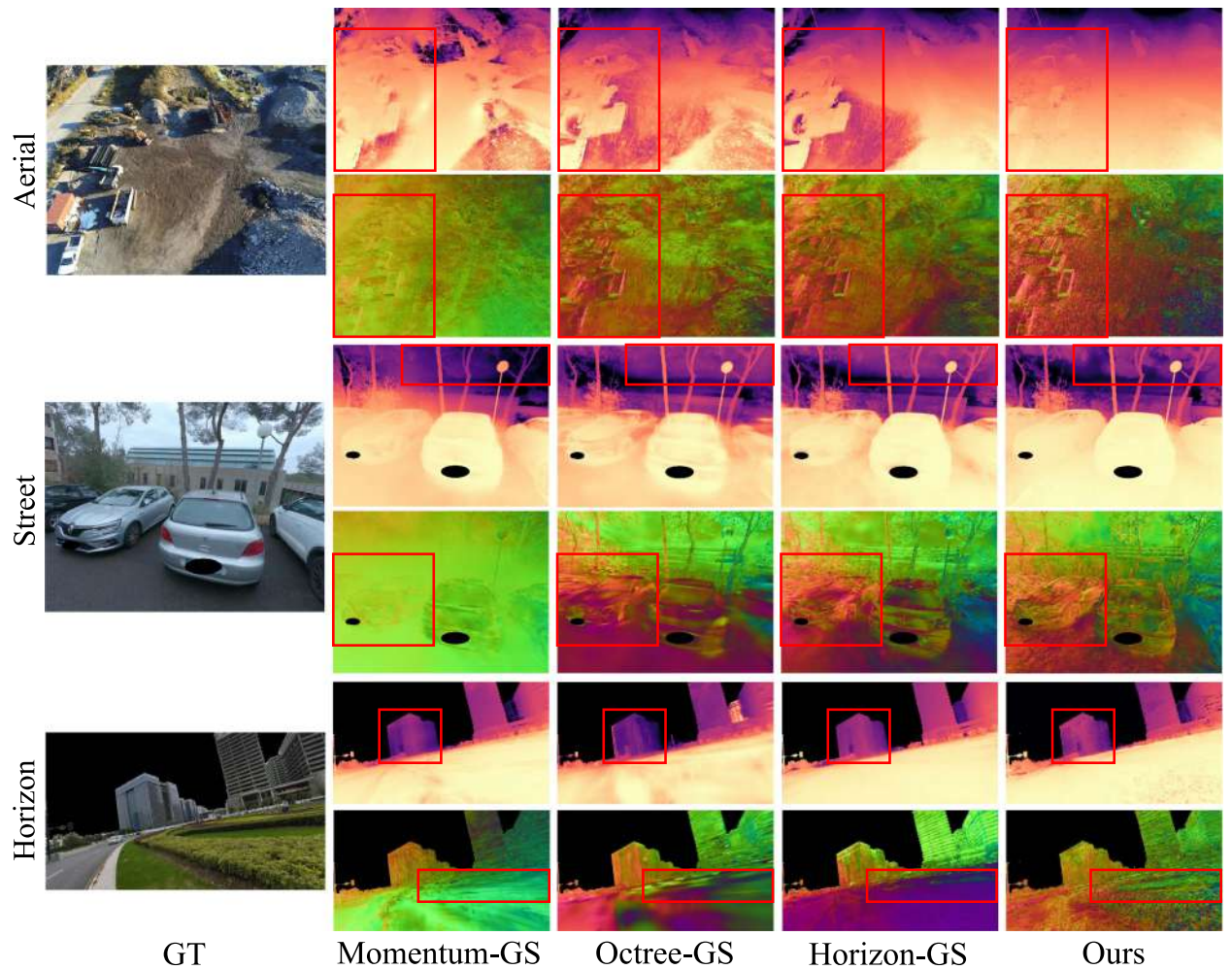


Fig. 7. Comparison of depth and normal rendering across aerial, street, and horizon datasets. Red boxes highlight regions of interest. Our method produces more complete depth maps and more accurate surface normals on aerial, street, and horizon scenes.

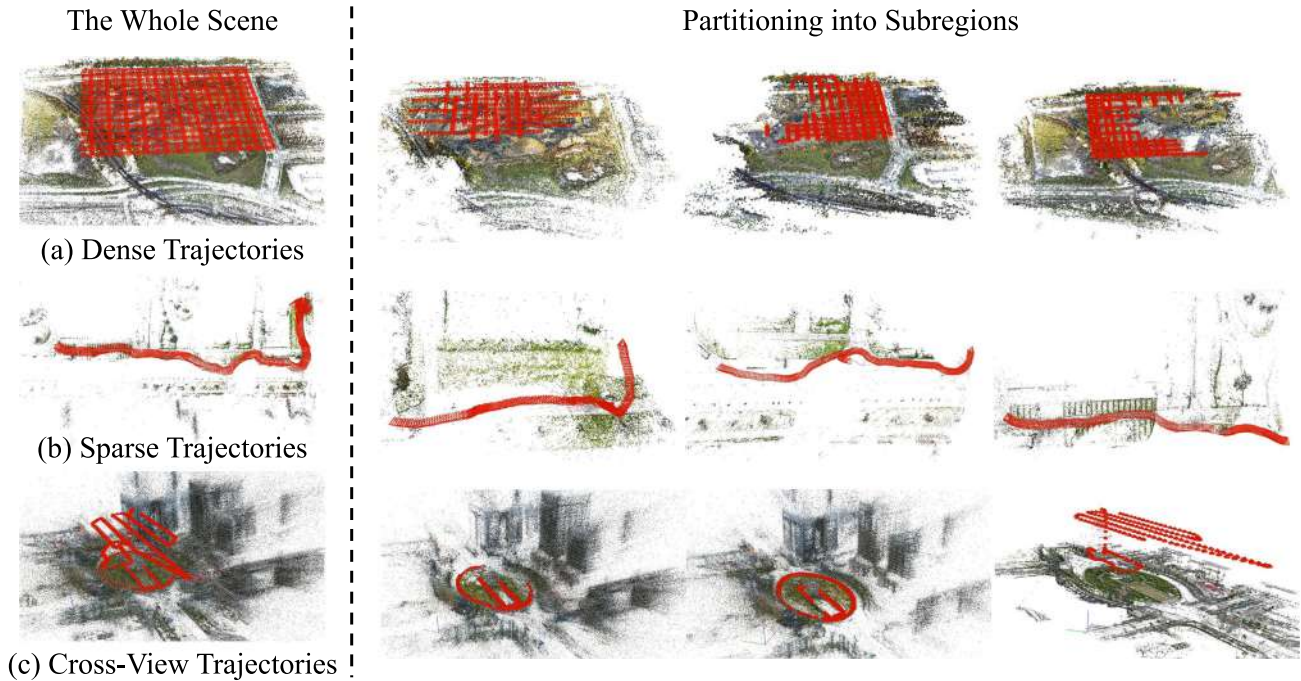


Fig. 8. Visualization of spatial partitioning based on our graph segmentation. (a) Partitioning cameras and point clouds of large-scale aerial scenes captured by dense camera trajectories into three spatial regions. (b) Partitioning cameras and point clouds of large-scale ground scenes captured by sparse camera trajectories into multiple spatial regions. Our adaptive spatial partitioning method is applicable across diverse scenes and camera trajectories. (c) Partitioning of cross-view datasets containing both aerial and ground-level imagery. Our method automatically segregates aerial and ground scenes based on co-visibility relationships, eliminating the manual intervention required by Horizon-GS and demonstrating superior flexibility and automation.

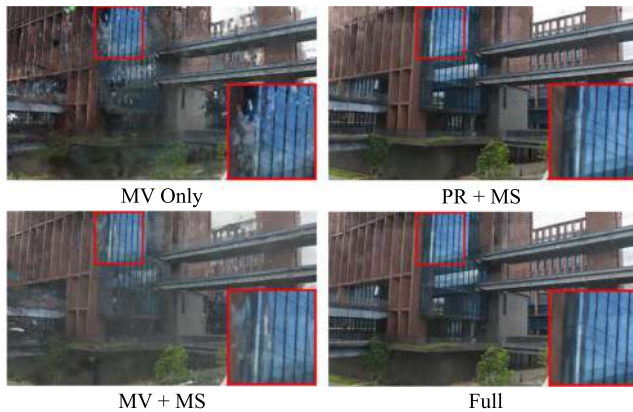


Fig. 9. Qualitative ablation comparison on a representative building scene. Red boxes highlight regions of interest with zoomed-in views.

It achieves 25.70 dB average PSNR, 0.792 SSIM, and 0.223 LPIPS, outperforming Horizon-GS by +0.44 dB, Hierarchy-GS by +0.94 dB, Momentum-GS by +2.10 dB, CityGaussian by +4.08 dB, and Octree-GS by +4.13 dB. Fig. 6 shows enhanced structural integrity and sharper details across cross-view scenes. Fig. 8 further visualizes the adaptive spatial partitioning results of our BFS-based graph segmentation method across different datasets, demonstrating its flexibility in handling dense, sparse, and cross-view camera trajectories.

TABLE IV
ABLATION STUDY RESULTS. PR, MV, AND MS DENOTE PROGRESSIVE RENDERING, MULTI-VIEW CONSTRAINT, AND MULTI-SCALE GAUSSIANS, RESPECTIVELY.

Configuration	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$
MV only	16.54	0.532	0.528
PR+MS	24.79	0.821	0.185
MV+MS	17.97	0.577	0.450
Full Model (PR+MV+MS)	25.01	0.826	0.180

F. Depth and Normal Analysis

As shown in Fig. 7, our multi-view constraint yields more complete depth maps and accurate normals on aerial and ground datasets. On cross-view datasets, Horizon-GS achieves better far-field depth due to its two-stage training, while our method demonstrates superior near-field quality.

G. Ablation Studies

To validate each module's contribution, we conduct ablation studies across all 10 scenes with four configurations (Table IV). Complete per-scene ablation results are provided in the supplementary material. Progressive Rendering (PR) is the most impactful module, with PR+MS improving over MV only by +8.25 dB PSNR, +0.289 SSIM, and -0.343 LPIPS. Note that progressive rendering introduces only a marginal overhead in rendering speed, since the coarse global Gaussian is intentionally lightweight (trained without densification on downsampled

TABLE V
EFFECT OF REGION NUMBERS ON RENDERING QUALITY AND GPU MEMORY

Scene	Regions	PSNR $\uparrow$	Aerial SSIM $\uparrow$	LPIPS $\downarrow$	PSNR $\uparrow$	Street SSIM $\uparrow$	LPIPS $\downarrow$	GPU Mem
Park	2	20.28	0.583	0.461	23.12	0.703	0.307	32930M
	4	20.86	0.626	0.405	24.29	0.737	0.267	24660M
	8	21.36	0.661	0.356	24.66	0.749	0.244	19800M
Citysample	2	31.40	0.926	0.120	24.02	0.808	0.252	25612M
	4	32.49	0.943	0.092	24.35	0.817	0.227	20498M
	8	33.96	0.953	0.081	24.32	0.823	0.221	17724M

TABLE VI
COMPARISON OF PARTITIONING METHODS ON RENDERING QUALITY AND GRAPH PROPERTIES

Method	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Balance	Connect.
<i>Ground-road</i>					
Spectral Decom.	26.77	0.849	0.136	$\times$	$\checkmark$
Kernighan-Lin	25.39	0.816	0.163	$\checkmark$	$\times$
Ours (BFS)	26.96	0.849	0.130	$\checkmark$	$\checkmark$
<i>Aerial-building</i>					
Spectral Decom.	22.83	0.799	0.200	$\times$	$\checkmark$
Kernighan-Lin	19.99	0.677	0.311	$\checkmark$	$\times$
Ours (BFS)	23.30	0.817	0.182	$\checkmark$	$\checkmark$

data) and each local Gaussian covers a small spatial extent with few primitives. A detailed speed analysis is provided in the supplementary material. Multi-Scale Gaussians (MS) contribute +1.43 dB PSNR (MV only vs. MV+MS), while Multi-View Constraint (MV) adds +0.22 dB on top of PR+MS, with more pronounced gains in texture-rich scenes. The Full Model achieves the best performance, confirming each module's additive contribution. Fig. 9 corroborates these findings: PR eliminates inter-region floating-point artifacts, MS improves far-field quality and visual coherence, and MV provides texture-level refinement.

We also investigate the effect of region count on the Horizon-GS datasets (Table V). Increasing regions consistently reduces GPU memory and improves quality: on *Park*, going from 2 to 8 regions improves aerial PSNR by 1.08 dB and street PSNR by 1.54 dB while reducing memory from 32,930 M to 19,800 M, as finer partitioning enables each local Gaussian to focus on a smaller spatial extent.

Additionally, we compare partitioning methods on two representative datasets (Table VI). Spectral decomposition preserves connectivity but produces imbalanced partitions, while Kernighan-Lin achieves balance but breaks connectivity, causing co-visibility fractures and significant quality drops (e.g., 19.99 dB vs. our 23.30 dB on Aerial-building). Our BFS-based method ensures both balance and connectivity, achieving the best metrics across both datasets.

V. CONCLUSION

In this paper, we propose TraGraph-GS, a trajectory graph-based method for high-quality novel view synthesis of arbitrarily large-scale scenes. Our graph-based spatial partitioning adaptively groups cameras by co-visibility, preserving multi-view consistency across diverse trajectories. Combined with regularization constraints for texture and distant-object enhancement,

and a progressive rendering strategy to suppress floating artifacts, TraGraph-GS achieves state-of-the-art rendering quality while generating high-quality depth and normal maps beneficial for downstream tasks. A current limitation, shared with prior works [10], [11], [12], is sensitivity to illumination variations. Future work will explore appearance encoding techniques to decouple and model lighting effects, reducing inter-image illumination inconsistencies.

REFERENCES

- [1] S. M. Seitz and C. R. Dyer, "View morphing," in *Proc. 23rd Annu. Conf. Comput. Graph. Interactive Techn.*, 1996, pp. 21–30.
- [2] C. Buehler, M. Bosse, L. McMillan, S. Gortler, and M. Cohen, "Unstructured lumigraph rendering," in *Proc. 28th Annu. Conf. Comput. Graph. Interactive Techn.*, 2001, pp. 425–432.
- [3] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, "NeRF: Representing scenes as neural radiance fields for view synthesis," *Commun. ACM*, vol. 65, no. 1, pp. 99–106, 2021.
- [4] J. T. Barron, B. Mildenhall, M. Tancik, P. Hedman, R. Martin-Brualla, and P. P. Srinivasan, "Mip-NeRF: A multiscale representation for anti-aliasing neural radiance fields," in *Proc. IEEE/CVF Int. Conf. Comp. Vis.*, 2021, pp. 5855–5864.
- [5] J. T. Barron, B. Mildenhall, D. Verbin, P. P. Srinivasan, and P. Hedman, "Mip-NeRF 360: Unbounded anti-aliased neural radiance fields," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 5470–5479.
- [6] C. Reiser, S. Peng, Y. Liao, and A. Geiger, "KiloNeRF: Speeding up neural radiance fields with thousands of tiny MLPs," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2021, pp. 14335–14345.
- [7] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, "3D Gaussian splatting for real-time radiance field rendering," *ACM Trans. Graph.*, vol. 42, no. 4, pp. 139–1, 2023.
- [8] Z. Yu, A. Chen, B. Huang, T. Sattler, and A. Geiger, "Mip-splatting: Alias-free 3D Gaussian splatting," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2024, pp. 19447–19456.
- [9] T. Lu et al., "Scaffold-GS: Structured 3D Gaussians for view-adaptive rendering," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2024, pp. 20654–20664.
- [10] Y. Liu, C. Luo, L. Fan, N. Wang, J. Peng, and Z. Zhang, "CityGaussian: Real-time high-quality large-scale scene rendering with Gaussians," in *Eur. Conf. Comput. Vis.*, 2024, pp. 265–282.
- [11] J. Lin et al., "VastGaussian: Vast 3D Gaussians for large scene reconstruction," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2024, pp. 5166–5175.
- [12] B. Kerbl, A. Meuleman, G. Kopanas, M. Wimmer, A. Lanvin, and G. Drettakis, "A hierarchical 3D Gaussian representation for real-time rendering of very large datasets," *ACM Trans. Graph. (TOG)*, vol. 43, no. 4, pp. 1–15, 2024.
- [13] X. Zhang, Z. Sun, Y. Qiu, J. Su, and Q. Liu, "Toy-GS: Assembling local Gaussians for precisely rendering large-scale free camera trajectories," in *Proc. AAAI Conf. Artif. Intell.*, vol. 39, no. 10, 2025, pp. 10212–10220.
- [14] X. Zhang, Y. Qiu, Z. Sun, and Q. Liu, "Aerial-NeRF: Adaptive spatial partitioning and sampling for large-scale aerial rendering," 2024, *arXiv:2405.06214*.
- [15] J. Fan, W. Li, Y. Han, T. Dai, and Y. Tang, "Momentum-GS: Momentum Gaussian self-distillation for high-quality large scene reconstruction," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2025, pp. 25250–25260.
- [16] L. Jiang et al., "Horizon-GS: Unified 3D Gaussian splatting for large-scale aerial-to-ground scenes," in *Proc. Comput. Vis. Pattern Recognit. Conf.*, 2025, pp. 26789–26799.
- [17] K. Zhang, G. Riegler, N. Snively, and V. Koltun, "NeRF++: Analyzing and improving neural radiance fields," 2020, *arXiv:2010.07492*.
- [18] S. Fridovich-Keil, A. Yu, M. Tancik, Q. Chen, B. Recht, and A. Kanazawa, "Plenoxels: Radiance fields without neural networks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 5501–5510.
- [19] A. Dhiman et al., "Strata-NeRF: Neural radiance fields for stratified scenes," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2023, pp. 17603–17614.

- [20] G. Wang, J. Zhang, K. Zhang, R. Huang, and L. Fang, "GiganticNVS: Gigapixel large-scale neural rendering with implicit meta-deformed manifold," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 1, pp. 338–353, 2024.
- [21] T. Wu, J.-M. Sun, Y.-K. Lai, and L. Gao, "Vd-NeRF: Visibility-aware decoupled neural radiance fields for view-consistent editing and high-frequency relighting," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 47, no. 5, pp. 3344–3357, 2025.
- [22] Z. Fan, K. Wang, K. Wen, Z. Zhu, D. Xu, and Z. Wang, "LightGaussian: Unbounded 3D Gaussian compression with 15x reduction and 200 fps," in *Proc. Adv. Neural Info. Process. Syst.*, vol. 37, pp. 140138–140158, 2024.
- [23] J. C. Lee, D. Rho, X. Sun, J. H. Ko, and E. Park, "Compact 3D Gaussian representation for radiance field," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2024, pp. 21719–21728.
- [24] T. Wu, J.-M. Sun, Y.-K. Lai, Y. Ma, L. Kobbelt, and L. Gao, "DeferredGS: Decoupled and relightable Gaussian splatting with deferred shading," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 47, no. 8, pp. 6307–6319, 2025.
- [25] M. Tancik et al., "Block-NeRF: Scalable large scene neural view synthesis," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 8248–8258.
- [26] K. Cheng et al., "GaussianPro: 3D Gaussian splatting with progressive propagation," in *Proc. 41 Int. Conf. Mach. Learn.*, 2024, pp. 8123–8140.
- [27] D. Luebke, M. Reddy, J. D. Cohen, A. Varshney, B. Watson, and R. Huebner, *Level of Detail for 3D Graphics*. Amsterdam, The Netherlands: Elsevier, 2002.
- [28] C. Dachsbacher, C. Vogelgsang, and M. Stamminger, "Sequential point trees," *ACM Trans. Graph.*, vol. 22, no. 3, pp. 657–662, 2003.
- [29] S. Rusinkiewicz and M. Levoy, "Qsplat: A multiresolution point rendering system for large meshes," in *Proc. 27th Annu. Conf. Comput. Graph. Interactive Techn.*, 2000, pp. 343–352.
- [30] K. Ren et al., "Octree-GS: Towards consistent real-time rendering with lod-structured 3D Gaussians," *IEEE Trans. Pattern Anal. Mach. Intell.*, early access, 2025, doi: [10.1109/TPAMI.2025.3568201](https://doi.org/10.1109/TPAMI.2025.3568201).
- [31] A. Meuleman, I. Shah, A. Lanvin, B. Kerbl, and G. Drettakis, "On-the-fly reconstruction for large-scale novel view synthesis from unposed images," *ACM Trans. Graph. (TOG)*, vol. 44, no. 4, pp. 1–14, 2025.
- [32] Z. Yan, W. F. Low, Y. Chen, and G. H. Lee, "Multi-scale 3D Gaussian splatting for anti-aliased rendering," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2024, pp. 20923–20931.
- [33] N. Snavely, S. M. Seitz, and R. Szeliski, "Photo tourism: Exploring photo collections in 3D," in *Proc. ACM Siggraph 2006 papers*, 2006, pp. 835–846.
- [34] S. Agarwal et al., "Building Rome in a day," *Commun. ACM*, vol. 54, no. 10, pp. 105–112, 2011.
- [35] H. Turki, D. Ramanan, and M. Satyanarayanan, "Mega-NeRF: Scalable construction of large-scale NeRFs for virtual fly-throughs," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 12922–12931.
- [36] M. Zhenxing and D. Xu, "Switch-NeRF: Learning scene decomposition with mixture of experts for large-scale neural radiance fields," in *Proc. 11th Int. Conf. Learn. Representations*, 2022.
- [37] K. Rematas et al., "Urban radiance fields," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 12932–12942.
- [38] Y. Liu, C. Luo, Z. Mao, J. Peng, and Z. Zhang, "CityGaussianV2: Efficient and geometrically accurate reconstruction for large-scale scenes," in *Proc. 13th Int. Conf. Learn. Representations*, 2025.
- [39] Y. Chen and G. H. Lee, "DOGS: Distributed-oriented Gaussian splatting for large-scale 3D reconstruction via Gaussian consensus," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 37, pp. 34487–34512, 2024.
- [40] D. G. Lowe, "Distinctive image features from scale-invariant keypoints," *Int. J. Comput. Vis.*, vol. 60, pp. 91–110, 2004.
- [41] J. L. Schonberger and J.-M. Frahm, "Structure-from-motion revisited," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 4104–4113.
- [42] M. Fiedler, "Algebraic connectivity of graphs," *Czechoslovak Math. J.*, vol. 23, no. 2, pp. 298–305, 1973.
- [43] B. W. Kernighan and S. Lin, "An efficient heuristic procedure for partitioning graphs," *The Bell Syst. Tech. J.*, vol. 49, no. 2, pp. 291–307, 1970.
- [44] A. Bundy and L. Wallen, "Breadth-first search," in *Proc. Catalogue Artif. Intell. Tools*, 1984, pp. 13–13.
- [45] C. Barnes, E. Shechtman, A. Finkelstein, and D. B. Goldman, "Patch-match: A randomized correspondence algorithm for structural image editing," *ACM Trans. Graph.*, vol. 28, no. 3, 2009, Art no. 24.
- [46] J.-C. Yoo and T. H. Han, "Fast normalized cross-correlation," *Circuits Syst. Signal Process.*, vol. 28, pp. 819–843, 2009.
- [47] Y. Chen, C. Gu, J. Jiang, X. Zhu, and L. Zhang, "Periodic vibration Gaussian: Dynamic urban scene reconstruction and real-time rendering," *Int. J. Comput. Vis.*, vol. 134, no. 3, 2026, Art. no. 83.
- [48] L. Lin, Y. Liu, Y. Hu, X. Yan, K. Xie, and H. Huang, "Capturing, reconstructing, and simulating: The UrbanScene3D dataset," in *Proc. Eur. Conf. Comput. Vis.*, 2022, pp. 93–109.



Xiaohan Zhang received the BS degree from Xinjiang University, Urumqi, China and the MSc degree from the School of Mathematics and Statistics, Shandong University, Weihai, China. He is currently working toward the PhD degree with the School of Future Technology, South China University of Technology. His research interests include 3D vision, including multi-view stereo, NeRF, 3DGS, and the application of 3D reconstruction technology in large-scale scenes.



Sitong Wang is currently working toward the bachelor's degree majoring in artificial intelligence with the School of Future Technology, South China University of Technology. His research interests include computer vision and computer graphics.



Yushen Yan is currently working toward the bachelor's degree with the South China University of Technology. His research interests include 3D vision and particularly 3D reconstruction technologies.



Yi Yang is currently working toward the master's degree with the School of Future Technology, South China University of Technology, China. His research interests include 3D vision, text-to-music generation, and text-guided music editing.



Mingda Xu received the BS degree from Xinjiang University, Urumqi, China, and the MSc degree from the Zhongtai Securities Institute for Financial Studies, Shandong University, China. She is currently working toward the PhD degree with the School of Future Technology, South China University of Technology. Her major is electronic information. Her research interests include 3D vision, including NeRF, 3DGS, and the application of 3D reconstruction technology in different scenes.



Qi Liu received the PhD degree in electrical engineering from the City University of Hong Kong, Hong Kong, SAR, China, in 2019. He is currently a professor with the School of Future Technology, South China University of Technology. From 2018 to 2019, he was a visiting scholar with the University of California Davis, CA, USA. From 2019 to 2022, he was a research fellow with the Department of Electrical and Computer Engineering, National University of Singapore, Singapore. He has authored or coauthored more than 50 papers in peer-reviewed journals and conferences, including *IEEE Transactions on Fuzzy Systems*, *IEEE Transactions on Cybernetics*, *IEEE Transactions on Circuits and Systems for Video Technology*, *IEEE Internet of Things Journal*, *ACL*, *CVPR*, and *AAAI*. His research interests include human-object interaction, AIGC, 3D scene reconstruction, and affective computing. He has been an associate editor of the *IEEE Systems Journal* since 2022, and *Digital Signal Processing* since 2022. He was also guest editor of the *IEEE Transactions on Consumer Electronics*, and *IEEE Internet of Things Journal*, *IET Signal Processing*. He was the recipient of the Best Paper Award from IEEE ICSIDP in 2019 and ICCBD+AI in 2024.